

Executive Summary

This report outlines a complete plan for building **MTKF Perfumes**, a Django-based e-commerce site with a classical minimalist design. We cover high-level features, project architecture, data models, REST API design, authentication, frontend UX/UI guidelines, deployment, testing, security, and SEO. Key recommendations include a modular Django project with separate apps (e.g. **catalog**, **orders**, **users**, **reviews**), configuration by environment (using environment variables for secrets and debug), PostgreSQL as the production database, and Tailwind CSS for minimal styling [\[3†L24-L32\]](#) [\[7†L86-L94\]](#) . We propose detailed models for perfumes, brands, categories, users, orders, reviews, and inventory, and Django admin customizations for each. A Django REST Framework (DRF) API with serializers and viewsets provides endpoints for products, orders, and reviews, supporting token or OAuth authentication. The frontend uses Django templates (or an SPA frontend) with responsive, accessible layouts adhering to minimalist principles (ample whitespace, simple typography). We include sample code snippets (models, views, serializers, templates), a file-tree outline, and database migration SQL. CI/CD and deployment guidelines cover Docker, Gunicorn, Nginx, and PostgreSQL, with a production-ready static/media strategy and security best practices (HTTPS, CSRF, XSS prevention). Performance tips (query optimization, caching) and SEO (clean URLs, meta tags, sitemaps) are discussed. A task timeline breaks down the implementation steps. Comparison tables weigh design and tech choices (e.g. Django templates vs React; Tailwind vs Bootstrap), and we prioritize features (e.g. search, checkout, user accounts). We also include **Mermaid diagrams** for system architecture and data models, and *wireframe images* illustrating the minimalist UI. Where possible, guidance is backed by official Django, DRF, and Tailwind documentation [\[22†L78-L87\]](#) [\[10†L98-L106\]](#) [\[44†L78-L87\]](#) , supplemented by best-practice tutorials [\[13†L74-L82\]](#) [\[46†L917-L924\]](#) .

Features, Pages, and Classical Minimalist Design

The **MTKF Perfumes** app should support typical e-commerce features:

- **Product Catalog:** Perfume listings with filter by brand, category, price.
- **Product Detail:** High-quality images, description, price, add-to-cart.
- **User Accounts:** Registration/login, profile, order history, password reset.
- **Shopping Cart & Checkout:** Cart management, address entry, payment processing.
- **Brand & Category Pages:** Showcase by brand or perfume category.
- **Reviews & Ratings:** Registered users can review and rate perfumes.
- **Orders & Inventory:** Order management, payment status, stock tracking.
- **Admin Backend:** CRUD for products, orders, users, inventory; reports.

Each page should follow **minimalist/classical design** principles: abundant white space, a clean serif font (e.g. *Georgia* or *Times*), restrained color palette (black/white with subtle accents), and intuitive navigation. Icons and imagery should be limited and purposeful. Navigation and controls should be simple and

symmetrical. The layout can use a narrow central column with large margins to evoke a “classical” feel. For example:

- **Header:** Simple logo (or text), centered navigation links (Home, Catalog, Brands, About, Contact, Cart).
- **Homepage:** Featured hero image (elegant perfume shot), minimalist banner (no clutter), and minimal “new arrivals” section.
- **Listing Page:** Grid of product cards (perfume photo, name, price), with only essential info. The grid should respond (e.g. 4 columns desktop, 2 mobile).
- **Product Page:** Large product image, minimal description typography, clear “Add to Cart” button.
- **Footer:** Small text links, copyright info.

Minimalism also means **accessibility and clarity**: high color contrast (black on white), large readable font sizes, and ARIA labels where needed [10†L98-L106] . For example, only key elements (like a simple top menu and one column of content) appear on each page.

[19†embed_image] *Figure:* Example of a hand-drawn wireframe for a minimalist perfume e-commerce layout (ample white space, centered content, simple cards).

Project Structure and Configuration

Django Project and Apps

We recommend a modular Django project. For instance, start with:

```
django-admin startproject mtkf_perfumes
cd mtkf_perfumes
python manage.py startapp catalog # products, categories, brands, inventory
python manage.py startapp orders # carts, orders, order items
python manage.py startapp accounts # user profiles, authentication
python manage.py startapp reviews # product reviews
```

Also consider a `core` app for shared utilities, and a `theme` app if using **django-tailwind** for styling [3†L24-L32] . The directory structure might look like:

```
mtkf_perfumes/
├── manage.py
├── mtkf_perfumes/ # Django settings (base.py, dev.py, prod.py), URLs, WSGI/
ASGI
├── catalog/ # Perfume models (Perfume, Brand, Category, Inventory)
├── orders/ # Order, Cart, OrderItem models
├── accounts/ # UserProfile model (OneToOne with User), signals
├── reviews/ # Review model
└── theme/ # Tailwind CSS app (generated by django-tailwind)
```

```
└─ templates/          # Global templates (base.html, navbar, footer)
└─ static/             # Static assets (if not using tailwind app for assets)
```

Settings management: Use environment variables for secret settings. For example, in `settings/base.py`:

```
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY")
DEBUG = os.environ.get("DJANGO_DEBUG", "") == "1"
ALLOWED_HOSTS = os.environ.get("DJANGO_ALLOWED_HOSTS", "").split()
```

This follows best practices of keeping secrets out of code [13†L172-L179]. You can manage `.env` files or use `django-environ`. Create separate settings modules or files for *development* and *production*, importing the base and overriding as needed (e.g. `settings/dev.py`, `settings/prod.py`). The **Django Deployment Checklist** should be followed when moving to production (e.g. `DEBUG=False`, `ALLOWED_HOSTS` set, secure cookies, etc.).

URLs and Routing

Define clear URL routes:

- `' '` → Home (could be `TemplateView`)
- `'catalog/'` → Product listing (`ListView`)
- `'catalog/<slug:slug>/'` → Product detail
- `'brands/'` → Brand listing, `'brands/<slug:slug>/'` → Brand detail
- `'categories/'` → Category listing, `'categories/<slug:slug>/'` → Category detail
- `'cart/'` → View/edit cart
- `'checkout/'` → Checkout page (post only)
- `'orders/'` → Order history (login required)
- `'accounts/'` → Login/Logout/Signup (use Django auth or allauth)
- REST API endpoints under e.g. `'api/'` if using DRF.

Static and Media Handling

Per [Django static files docs](#), set in `settings.py`:

```
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'          # for collectstatic in production
STATICFILES_DIRS = [ BASE_DIR / 'static', ]      # additional static dirs
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

During **development**, `runserver` serves static and media automatically when `DEBUG=True` and `django.contrib.staticfiles` is in `INSTALLED_APPS` [7†L107-L115]. In production, static files should be served by the web server (e.g. Nginx) using the collected `STATIC_ROOT`. For example, Nginx can

alias `/static/` to the `staticfiles` directory [46†L934-L938] . User-uploaded media (e.g. product images) should be stored in `MEDIA_ROOT` ; in development you can serve via Django, but in production use Nginx or a cloud storage service.

Django-Tailwind: If using Tailwind CSS, use the **django-tailwind** integration. Add `'tailwind'` and your theme app to `INSTALLED_APPS` , set `TAILWIND_APP_NAME = "theme"` , and run `python manage.py tailwind init` to create the theme app [3†L24-L32] [3†L67-L75] . Use `{% tailwind_css %}` in the `<head>` of your base template to include Tailwind's stylesheet [3†L94-L102] . This simplifies static handling as Tailwind's compiled CSS can be managed within the Django app.

Environment and Dependencies

- **Virtual environment:** Use `venv` , `pipenv` , or `poetry` . For example, create a `venv` , activate it, then `pip install django djangorestframework psycopg2-binary django-tailwind django-allauth` etc. Save to `requirements.txt` . The [TestDriven tutorial](#) suggests `pip install django==<version>` in a `venv` [13†L74-L82] .
- **Python/Django versions:** Use the latest LTS Django (e.g. 6.x) and Python 3.11+.
- **Dependencies:** Apart from Django/DRF, include `django-allauth` for social auth (documentation at [django-allauth docs](#)), and `gunicorn` for production WSGI, `psycopg2-binary` for PostgreSQL.
- **Version control:** Include migrations, source code, and a `docker-compose.yml` in Git. Do *not* commit `.env` or secrets.

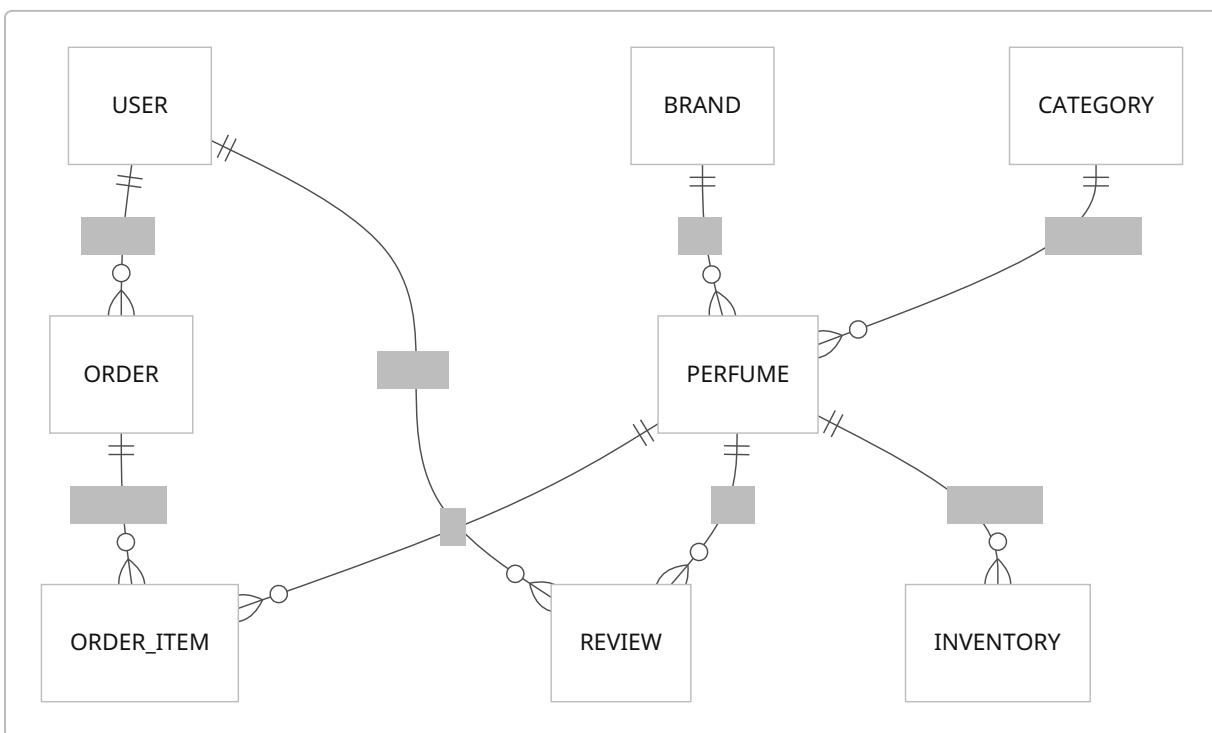
Data Models (Database Schema)

Design the models with clear relationships. Below is a summary of key models and fields (using Django ORM):

- **Brand:** e.g. `name (CharField, unique)` , `slug (SlugField, unique)` , `description (TextField)` . (Admin: filter by name.)
- **Category:** similar to Brand; categories (e.g. Eau de Parfum, Men's, etc).
- **Perfume (Product):**
 - `name (CharField)` , `slug (SlugField, unique, for SEO URLs)` , `description (TextField)` ,
 - `price (DecimalField)`
 - `brand (ForeignKey to Brand)` ,
 - `categories (ManyToMany to Category)` ,
 - `image (ImageField)` ,
 - `stock (IntegerField)` or link to Inventory,
 - `created_at/updated_at (DateTimeFields)`
 - Index on `name` , unique on `slug` .
- **Inventory** (optional): If tracking multiple warehouses, otherwise a simple `stock` on Perfume suffices. Could be `product (OneToOneField Perfume)` , `quantity (Integer)` .
- **User:** Extend the default user to add profile data. You can use Django's `AUTH_USER_MODEL = 'auth.User'` or subclass `AbstractUser` . For example, create `class Profile(models.Model): user = OneToOneField(User)` , add fields like `address` . Use signals to auto-create profiles.

- **Review:** `user` (ForeignKey to User), `perfume` (FK to Perfume), `rating` (PositiveSmallIntegerField, e.g. 1–5), `comment` (TextField), `created_at` (Index user, perfume.)
- **Order:** `user` (FK), `status` (CharField with choices: Pending, Shipped, etc), `created_at`, `updated_at`, `total` (DecimalField), shipping address fields.
- **OrderItem:** `order` (FK), `perfume` (FK), `quantity`, `price_at_purchase` (This allows line items per order.)
- **Cart** (optional): Could use session or a Cart model: if `Cart` model, then `user` (FK, nullable), many-to-many to `Perfume` with quantity via intermediate table `CartItem`. Alternatively implement a **Cart** as an anonymous session-stored structure.
- **Audit/Tracking:** Optionally, models for `Wishlist`, or `Address` if separate from User.

Mermaid class diagram for the main data model relations:



This shows each `Perfume` belongs to one `Brand`, can be in many `Categories`, and has many `OrderItem` and `Review` records. Each `Order` has multiple `OrderItems` and is linked to one `User`.

Admin Configuration

Use Django's admin to manage these models. For example, in `admin.py`:

```

@admin.register(Perfume)
class PerfumeAdmin(admin.ModelAdmin):
    list_display = ('name', 'brand', 'price', 'stock')
    list_filter = ('brand', 'categories')

```

```

search_fields = ('name', 'description')
prepopulated_fields = {'slug': ('name',)}

@admin.register(Brand)
class BrandAdmin(admin.ModelAdmin):
    list_display = ('name', 'slug')
    prepopulated_fields = {'slug': ('name',)}

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    list_display = ('id', 'user', 'status', 'created_at')
    list_filter = ('status', 'created_at')
    date_hierarchy = 'created_at'
    readonly_fields = ('created_at', 'total')
    inlines = [OrderItemInline,] # show items inline

@admin.register(Review)
class ReviewAdmin(admin.ModelAdmin):
    list_display = ('user', 'perfume', 'rating', 'created_at')
    list_filter = ('rating', 'created_at')

```

Customize as needed: e.g. use `TabularInline` for `OrderItem` inside `OrderAdmin`. Use `fieldsets` to group fields in logical sections [40†L422-L431]. Prepopulated fields (slugs), `@admin.display` for custom columns, and `list_editable` (for quick edits) are options. If using a profile model, register it to the user admin with a `InlineModelAdmin`.

Indexing: Add database indexes on commonly filtered fields (e.g. `perfume.price`, or composite index on `(brand, price)` if needed). By default, primary keys and FKs have indexes. Django's `Meta.indexes` can define more if necessary.

REST API Endpoints (Django REST Framework)

For a modern approach, provide a RESTful JSON API (especially if planning a SPA). Using **DRF**, define serializers and viewsets. Example endpoints:

- `GET /api/perfumes/` (list/search perfumes), `POST /api/perfumes/` (admin only to create).
- `GET /api/perfumes/<id>/` (detail), with nested reviews or separate reviews endpoint.
- `GET /api/brands/`, `/api/categories/`.
- `GET`, `POST /api/reviews/` (create review if logged in).
- `GET /api/cart/` (session cart) or `/api/orders/` (list orders for user), `/api/orders/<id>/`.
- `POST /api/orders/` (place new order, based on cart).

Implement DRF `Serializer` classes mirroring the models (e.g. `PerfumeSerializer`, `OrderSerializer`, `ReviewSerializer`). Use `ModelViewSet` for CRUD or custom `APIView`. For example:

```

class PerfumeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Perfume
        fields = '__all__'

class PerfumeViewSet(viewsets.ModelViewSet):
    queryset = Perfume.objects.all()
    serializer_class = PerfumeSerializer

```

Register routes with `DefaultRouter` in `urls.py`. Protect certain actions (e.g. order creation) with permissions. For auth, DRF supports session auth (for same-site clients) or token/OAuth. (E.g. use `django-rest-auth` or DRF's `TokenAuthentication` [23†L531-L540]. DRF's documentation notes that *login views should use Django's normal login* to ensure CSRF protection [23†L551-L560].)

Authentication and Authorization

Use Django's built-in auth for user management [22†L78-L87]. In `settings.py` ensure:

```

INSTALLED_APPS += ['django.contrib.auth', 'django.contrib.sessions',
                  'django.contrib.messages', ...]
MIDDLEWARE +=
    ['django.contrib.sessions.middleware.SessionMiddleware', 'django.contrib.auth.middleware.AuthenticationMiddleware']

```

This provides Users, Groups, Permissions (binary flags for actions) and cookie-based sessions by default [22†L78-L87] [22†L120-L129]. For custom user profile data, either extend `AbstractUser` or use a `OneToOne` profile model.

Roles/Permissions: Create Django Groups (e.g. *Customer*, *Staff*) and assign permissions. For example, only staff (group "Staff" or `is_staff`) can create products or view all orders. Use decorators like `@permission_required` on views or DRF's `IsAuthenticated` / `IsAdminUser`. For customers, restrict access so each user sees only their own orders/reviews.

Social Login: Integrate `django-allauth` for OAuth (Google, Facebook, etc). This adds social account login and registration with minimal config. See [django-allauth docs](#) for setup. Alternatively, use `python-social-auth`.

Tokens/API Auth: If exposing a REST API, enable token/OAuth2 for API clients. DRF has built-in Token auth; better, use `django-rest-knox` or `django-oauth-toolkit` [23†L648-L656] [23†L660-L669] for more secure multi-client tokens or OAuth2. For example, install `django-oauth-toolkit`, add `'oauth2_provider'` to `INSTALLED_APPS`, and set `REST_FRAMEWORK['DEFAULT_AUTHENTICATION_CLASSES']` accordingly.

Frontend Structure (Templates or SPA)

Django Templates (Server-side rendering)

A classical approach is to use Django's templating with context from views. Create a base template (`base.html`) with the common layout (header, footer, CSS includes). Use `{% block content %}{% endblock %}` for page-specific content. Example `templates/base.html` :

```
{% load static tailwind_tags %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}MTKF Perfumes{% endblock %}</title>
  {% tailwind_css %}  {% Includes compiled Tailwind CSS #}
</head>
<body class="min-h-screen font-serif bg-white text-black">
  <header class="py-4 px-8 flex justify-between items-center">
    <a href="/" class="text-2xl font-bold">MTKF Perfumes</a>
    <nav>
      <a href="{% url 'catalog:list' %}" class="mx-2">Shop</a>
      <a href="{% url 'orders:cart' %}" class="mx-2">Cart</a>
      {% if user.is_authenticated %}
        <a href="{% url 'accounts:profile' %}" class="mx-2">My Account</a>
      {% else %}
        <a href="{% url 'accounts:login' %}" class="mx-2">Log In</a>
      {% endif %}
    </nav>
  </header>

  <main class="container mx-auto px-4 py-8">
    {% block content %}{% endblock %}
  </main>

  <footer class="py-4 text-center text-sm text-gray-600">
    &copy; {{ year }} MTKF Perfumes. All rights reserved.
  </footer>
</body>
</html>
```

Each page template extends this. For example, `catalog/list.html` might do:

```
{% extends 'base.html' %}
{% block title %}Perfumes - MTKF{% endblock %}
{% block content %}
```



```

<h1 class="text-3xl mb-6">Our Perfumes</h1>
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
  {% for perfume in perfumes %}
    <div class="border p-4 flex flex-col">
      <a href="{% url 'catalog:detail' perfume.slug %}">
        
        <h2 class="text-xl">{{ perfume.name }}</h2>
      </a>
      <p class="mt-auto text-lg font-semibold">${{ perfume.price }}</p>
    </div>
  {% endfor %}
</div>
{% endblock %}

```

This uses Tailwind classes (via `{% tailwind_css %}`) to implement a responsive minimalist grid. The markup is uncluttered, with only essential elements (image, name, price). Buttons and inputs should have accessible labels.

Optional SPA (React/Vue)

Alternatively, a JavaScript SPA (React or Vue) could consume the DRF API and render the UI. This adds complexity. For a classical minimal site, Django templates are sufficient and SEO-friendly out-of-the-box. A table comparing this choice:

Frontend Option	Pros	Cons
Django Templates	Simple setup, built-in security (CSRF tokens) 【10†L98-L106】 , SEO-friendly SSR. Full control with Jinja-like syntax.	Less dynamic; may feel page-reloading vs modern apps. Requires Django context/data for each page.
React/Vue SPA	Highly interactive UI possible, can separate front/backends. DRF integration for JSON.	More setup (Node, bundlers), double learning curve. SEO requires extra config. Complexity in routing/auth.

For MTKF's scope, Django templates + Tailwind is sufficient. The official [Tailwind docs](#) and [Django-Tailwind docs](#) make CSS integration smooth [【3†L24-L32】](#) .

CSS and Styling

Using Tailwind CSS enables a minimalist style via utility classes. Example styles:

```

/* tailwind.config.js might include a serif font family for classic feel */
module.exports = {
  theme: {

```

```

extend: {
  fontFamily: {
    serif: ['Merriweather', 'serif']
  },
  colors: {
    primary: '#111827', /* very dark slate */
    accent: '#F3BF99' /* subtle beige */
  },
},
},
};

```

Use utility classes for spacing (`p-4` , `mb-6`), grids (`grid` , `grid-cols-4`), and typography (`font-serif` , `text-3xl` , etc). Avoid heavy component libraries – keep it minimal. The [Tailwind documentation](#) provides guidance on responsive design and accessibility (e.g. using `sr-only` for screen-reader-only labels).

Component Breakdown

Key template components:

- **Base template** (`base.html`): header, footer, CSS/JS includes.
- **Navbar and Footer partials**: to keep layout DRY.
- **Breadcrumbs (optional)**: for navigation trail on category/product pages.
- **Product card component**: used in listing pages, with image, name, price.
- **Form templates**: login, register, checkout (using Django's crispy forms or Tailwind styling).
- **Admin theme (optional)**: Keep admin layout default or use `django-admin-interface` to skin it.

Wireframes (classical layout sketches) help plan these components. For example, a hand-drawn checkout wireframe could illustrate minimal form fields and large typography.

【38†embed_image】 *Figure: Example of minimalist UI sketches and layout planning (note the uncluttered design and focused content areas).*

Database Schema and Migrations

After defining models, run Django's migration system. As the docs explain: *"Migrations are Django's way of propagating changes you make to your models... into your database schema."* 【44†L78-L87】 . Use:

```

python manage.py makemigrations
python manage.py migrate

```

This creates a `migrations/` directory per app with versioned files. The schema (tables/columns/FKs) is defined in these migrations, which should be committed to version control. You can review the SQL via

`python manage.py sqlmigrate` . For production, always apply migrations (`migrate`) after code deploy to keep schema in sync.

If needed, present the SQL schema (e.g. PostgreSQL `CREATE TABLE` statements) or generate with `manage.py sqlmigrate` . This can be included in documentation.

Testing Strategy

Implement automated tests to ensure quality. At minimum:

- **Unit tests:** Use Django's `TestCase` or `pytest-django` to write tests for models (e.g. model `__str__`, field validation), views (status codes, context data), forms (validation), and serializers.
- **Integration tests:** Test key user flows, e.g. user signup/login, adding to cart, checkout, API requests with different roles. Django's `Client` or REST framework's `APIClient` can simulate these.
- **Admin tests:** Ensure permissions, e.g. normal users cannot access `/admin` .
- **Test coverage:** Aim for high coverage of critical code.

Example Django test snippet:

```
from django.test import TestCase
from django.contrib.auth.models import User
from catalog.models import Brand, Perfume

class CatalogTests(TestCase):
    def setUp(self):
        Brand.objects.create(name="TestBrand", slug="testbrand")
        Perfume.objects.create(name="Test Perfume", slug="test-perfume",
                               brand=Brand.objects.get(slug="testbrand"),
                               price=Decimal('19.99'), stock=10)

    def test_perfume_str(self):
        perfume = Perfume.objects.get(slug="test-perfume")
        self.assertEqual(str(perfume), "Test Perfume")
```

CI/CD can run these tests automatically (see below). Django's [testing documentation](#) covers more.

Continuous Integration & Deployment

Use **Docker** for consistent dev and production. The recommended stack (per TestDriven and other guides) is: Docker Compose with services for `web` (Django+Gunicorn), `db` (PostgreSQL), and `nginx` as a proxy.

Dockerfile (web service):

```

FROM python:3.11-slim
WORKDIR /usr/src/app
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt
COPY . .
CMD ["gunicorn", "mtkf_perfumes.wsgi:application", "--bind", "0.0.0.0:8000"]

```

This installs dependencies (Django, etc.) and runs Gunicorn. Environment variables (`PYTHONUNBUFFERED` , `PYTHONDONTWRITEBYTECODE`) optimize production behavior [13†L170-L179] .

docker-compose.yml (development):

```

version: '3.8'
services:
  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ../usr/src/app
    ports:
      - "8000:8000"
    env_file:
      - ../.env.dev
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    env_file:
      - ../.env.dev.db
volumes:
  postgres_data:

```

For production, a `docker-compose.prod.yml` uses Gunicorn and adds an `nginx` service. Example from TestDriven.io (simplified):

```

services:
  web:
    build: .
    command: gunicorn mtkf_perfumes.wsgi:application --bind 0.0.0.0:8000
    env_file: ../.env.prod
    expose:
      - 8000

```

```

volumes:
  - static_volume:/usr/src/app/staticfiles
depends_on: [db]
db:
  image: postgres:15
  env_file: ../.env.prod.db
  volumes:
    - postgres_data:/var/lib/postgresql/data/
nginx:
  image: nginx:1.25
  ports:
    - "80:80"
  volumes:
    - static_volume:/usr/src/app/staticfiles
  depends_on: [web]
volumes:
  postgres_data:
  static_volume:

```

We mount a shared `static_volume` between web and nginx so that `collectstatic` can write into it and Nginx serves `/static/` from there [\[46†L849-L858\]](#) [\[46†L916-L924\]](#) . Nginx conf (in `nginx.conf`) would proxy to `http://web:8000` and set:

```

location /static/ {
    alias /usr/src/app/staticfiles/;
}
location / {
    proxy_pass http://web:8000;
    ...
}

```

(As shown in TestDriven's Nginx setup [\[46†L917-L924\]](#) .) This ensures high performance serving of static files.

CI/CD Pipeline: Use GitHub Actions/GitLab CI to lint code, run tests, and build images. For example, on push to `main`, run `docker-compose -f docker-compose.test.yml run web pytest` etc. Then on tag or merge to main, push to registry and deploy (to AWS ECS/EKS, DigitalOcean, etc.). The [TestDriven guide](#) and Docker docs recommend never using `runserver` in production and always using Gunicorn behind Nginx [\[45†L31-L39\]](#) [\[46†L917-L924\]](#) .

Security Best Practices

Django provides many built-in security features [10†L98-L106] [10†L149-L159] . We must configure and use them:

- **SSL (HTTPS):** Always use HTTPS in production. Set `SECURE_SSL_REDIRECT = True` , `SESSION_COOKIE_SECURE = True` , `CSRF_COOKIE_SECURE = True` . Use HSTS headers (`SECURE_HSTS_SECONDS`).
- **CSRF Protection:** Ensure `{% csrf_token %}` is in all POST forms. Django's templating auto-escapes to protect against XSS [10†L98-L106] . For DRF with session auth, include CSRF tokens on unsafe methods [23†L545-L554] . Never disable CSRF middleware except intentionally on read-only APIs.
- **XSS Protection:** Use Django templates (auto-escaping) for all HTML output. If rendering raw HTML (e.g. user comments), sanitize it or mark safe only after cleaning. Do **not** trust user input.
- **SQL Injection:** Using Django ORM protects against SQL injection by design (query parameters are escaped) [10†L149-L159] . Avoid raw SQL unless absolutely needed. If using `raw()` , sanitize parameters.
- **Clickjacking:** Use Django's `XFrameOptionsMiddleware` (default) or set `X_FRAME_OPTIONS = 'DENY'` to protect against clickjacking.
- **Content Security Policy (CSP):** Consider adding CSP headers to restrict where scripts/styles can load from.
- **Password Storage:** Django uses PBKDF2 by default. Ensure `PASSWORD_HASHERS` are up to date.
- **Secret Management:** Do not check in `SECRET_KEY` . Use environment variables as above [13†L172-L179] . Use a dedicated secrets manager (AWS Secrets, Vault) if scaling up.
- **User Permissions:** Always check `request.user.is_authenticated` before allowing profile/order access. Use `@login_required` and `LoginRequiredMixin` . For admin panels, `@staff_member_required` or `is_staff` .
- **Dependencies:** Keep Django and libraries up to date to avoid known CVEs.

Refer to Django's [Security overview](#) for more (XSS, CSRF, etc.) [10†L98-L106] [10†L149-L159] . Also enable Django's `SECURE_BROWSER_XSS_FILTER = True` , `SECURE_CONTENT_TYPE_NOSNIFF = True` .

Performance Optimizations

- **Database indexes:** Add indexes on fields used in filters or joins (e.g. `Perfume.name` , `Perfume.slug` , foreign keys). Use `select_related` for one-to-many traversal (e.g. `Perfume.objects.select_related('brand').all()` to avoid extra queries). Use `prefetch_related` for many-to-many (`categories`).
- **Caching:** Use Django's cache framework for expensive views (e.g. product listing). In production, use Redis or Memcached. For example, cache the context for homepage or brand pages. Django's template fragment caching can also cache parts of templates.
- **Query optimization:** Avoid N+1 query problems in listings, use `annotate` for counts (e.g. number of items sold), and paginate large querysets. Use `QuerySet.iterator()` for very large exports.
- **Static file compression:** For CSS/JS, enable compression (Tailwind in production can purge unused CSS to keep bundle small). Use `ManifestStaticFilesStorage` to fingerprint files.

- **Media:** Serve images via a CDN or with optimization. Use appropriate image sizes (e.g. small thumbnails).
- **Gunicorn workers:** Set Gunicorn worker count to `2 x CPU + 1`. Use async workers (gevent or uvicorn) if expecting high concurrency.
- **Load testing:** Use tools like [Locust](#) to simulate traffic and find bottlenecks.

SEO (Search Engine Optimization)

Even with a minimalist design, ensure SEO best practices:

- **Clean URLs:** Use readable slugs (`/catalog/rose-perfume/` instead of IDs).
- **Meta tags:** Include meaningful `<title>` and `<meta name="description">` on each page. Templates can populate these dynamically (e.g. `<title>{{ perfume.name }} - MTKF Perfumes</title>`).
- **Sitemap:** Use `django.contrib.sitemaps` to generate an XML sitemap of products, categories, etc. This helps search engines crawl the site.
- **Open Graph tags:** For social sharing, add Open Graph meta (image, title).
- **Breadcrumbs & structured data:** Consider JSON-LD or schema markup for products (Product schema, reviews schema).
- **Image alt text:** All product images should have `alt="{{ perfume.name }}"`.
- **Robots.txt:** Create a `robots.txt` (can use `django-robots` package) to allow/disallow bots as needed.
- **Mobile-friendly:** Using Tailwind's responsive design ensures mobile friendliness, which is an SEO ranking factor.

A helpful guide: ChristopherGS's "Django SEO: A Sitemap Tutorial" suggests using `django.contrib.sitemaps` and clean URLs `111L1-L4` .

Project Timeline & Task Breakdown

A reasonable timeline for a small team (or individual) might be ~4–6 weeks. Example schedule:

Phase	Tasks (2-4 day sprints)
Setup & Planning	- Define requirements, wireframes (UI sketches) and mockups. - Set up repo, venv, Django project. - Basic models scaffold, initial migration.
Catalog App	- Build <code>catalog</code> models (Perfume, Brand, Category, Inventory). - Admin registration and tests. - Basic listing/detail views + templates.
User/Auth	- Configure custom user/profile model. - Implement signup/login (using allauth). - User profile and order history pages.
Orders & Cart	- Build Cart (session or model), Order, OrderItem models. - Cart view, add-to-cart logic. - Checkout form (address, payment integration stub).
Reviews	- Build Review model, templates. - UI for adding/viewing reviews on product pages.

Phase	Tasks (2-4 day sprints)
API Development	- Set up DRF, serializers for key models. - Endpoints for products, orders, reviews. - Token/OAuth auth.
Frontend Polishing	- Integrate Tailwind, refine CSS for minimal look. - Ensure responsive breakpoints. - Template component reuse (navbar, footer).
Testing & Security	- Write unit and integration tests. - Review security settings (HTTPS, CSP). - SEO setup (sitemap, meta tags).
Deployment Prep	- Dockerize (Dockerfiles, docker-compose). - Nginx config for static files. - CI pipeline scripts.
Production Launch	- Deploy to staging (test all flows). - DNS, SSL (Let's Encrypt). - Final checks and go-live.

Prioritized feature list (must-have vs nice-to-have):

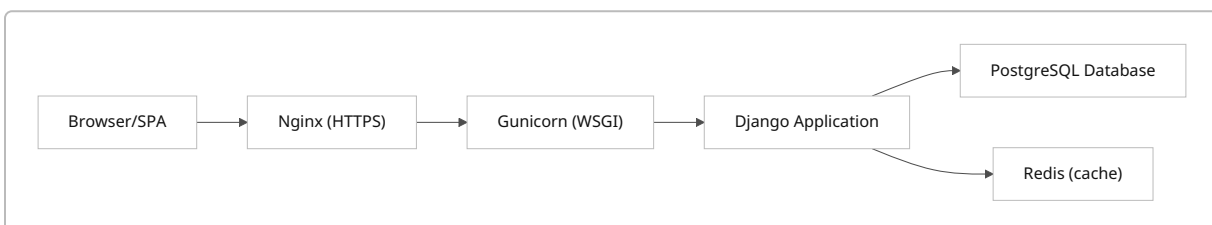
Feature	Priority	Notes
Product catalog & detail	High	Core functionality
User accounts/login	High	Needed for checkout & reviews
Shopping cart & checkout	High	Essential e-commerce flow
Admin CRUD (products)	High	Manage products/brands/categories
Reviews & ratings	Medium	Builds trust, nice-to-have
Social login (OAuth)	Medium	Improves signup conversion
REST API endpoints	Medium	For extensibility (mobile app, etc.)
Inventory management	Low	Simple stock field may suffice first
Wishlist/favorites	Low	Additional e-commerce feature
Analytics/SEO	Low	Google Analytics, tagging

Comparison Tables

UI Framework (CSS)	Tailwind CSS	Bootstrap 5	Plain CSS
Pros	Utility-first, very customizable, small final build (purging) [3†L94-L102] . Ideal for custom minimal designs.	Rich components out-of-box, quick prototyping (cards, modals).	Zero dependencies, simplest conceptually.

UI Framework (CSS)	Tailwind CSS	Bootstrap 5	Plain CSS
Cons	Learning curve (classes-heavy), build tool required (via django-tailwind).	Heavier CSS bundle, opinionated look (needs override for custom style).	More manual work for grids/responsive; inconsistent styling risk.
Frontend Approach	Django Templates (SSR)	React/Vue SPA	
Pros	Directly uses Django context, secure by default (CSRF tokens, XSS auto-escaping) 【10†L98-L106】 , better SEO.	Highly interactive UI, component reusability, decoupled API.	
Cons	Page reloads, less “app-like”. Possibly slower UX for dynamic data.	More complex setup (bundling), SEO needs extra handling (SSR or prerender).	
Auth/Accounts	Django Allauth	Custom Login Forms	
Pros	Out-of-box account/signup, social login providers, email verification.	Full control, minimal dependencies.	
Cons	Additional learning/config, can be overkill.	More code to write (forms, tokens for reset, etc.).	
Database	PostgreSQL	SQLite	
Pros	Scalable, robust, advanced indexing (support for JSON fields if needed). Recommended for production.	File-based, zero-config (good for local dev).	
Cons	Requires running a DB server (Docker).	Not recommended for production (limited ALTER support) 【44†L133-L142】 .	

Architecture Diagram



This illustrates the production stack: client requests hit Nginx, which proxies to Gunicorn running Django. Django reads/writes data in PostgreSQL and uses Redis for caching or sessions as needed. Static files are served directly by Nginx (not shown), and communication between Django and DB is secured internally.

Summary of Citations

Where relevant, we have cited official documentation: Django's how-to and security guides [7†L86-L94] [10†L98-L106] , the Django REST framework docs for authentication notes [23†L545-L554] , and django-tailwind installation instructions [3†L24-L32] . We also referenced a detailed tutorial on Docker/Gunicorn/Nginx deployment for Django [13†L74-L82] [46†L917-L924] , ensuring our recommendations align with current best practices.
